CBS COPENHAGEN BUSINESS SCHOOL
HANDELSHØJSKOLEN

LANGUAGE MODELING

N-GRAM

Rajani Singh
Assistant Professor, DIGI

EQUIS ACCREDITED AACSB ACCREDITED ASSOCIATION OF MBAS ACCREDITED CEMS PIM

Agenda

Language Model

- Understanding N-gram
- Building a language model
- Evaluation

Probability

- Conditional probability and chain rule
- Markov Assumption

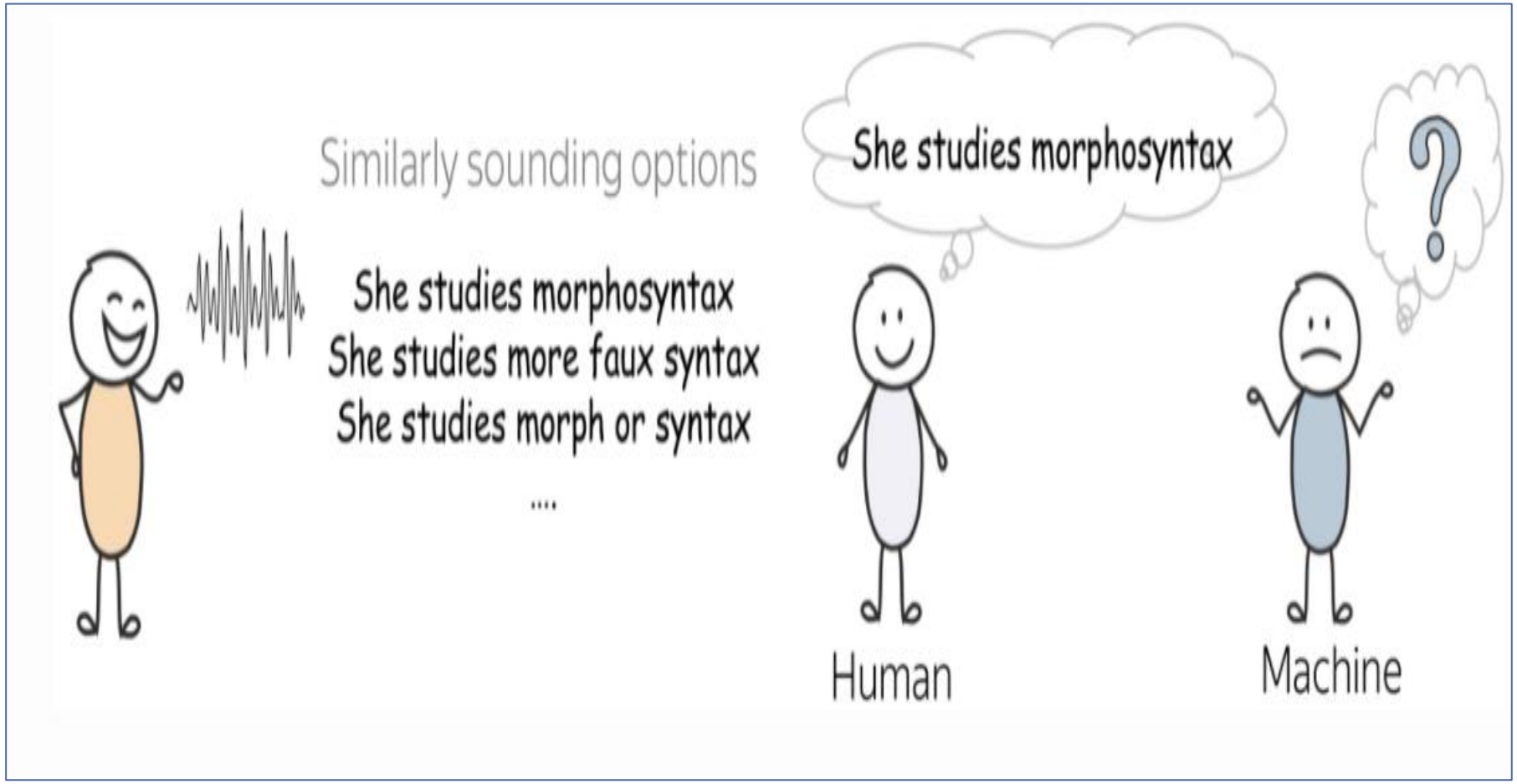
Issues and solution in LM

- Laplace Smoothing
- Interpolation

CBS

LANGUAGE MODEL (LM)

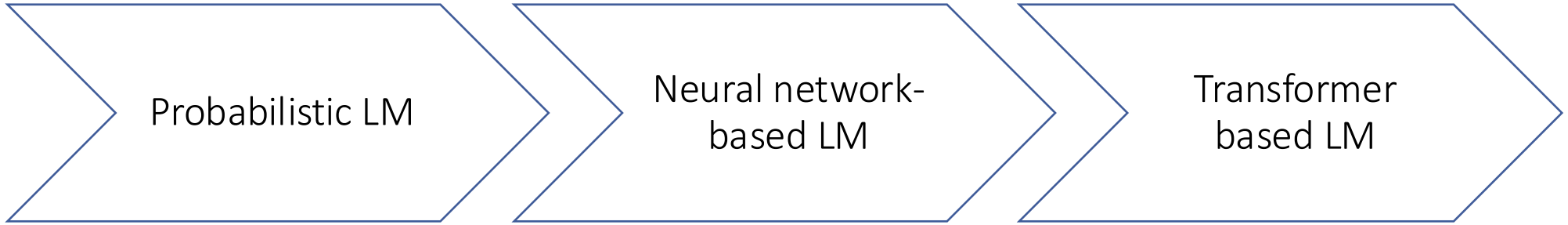
What is easy for humans, can be very hard for machines



CBS

LANGUAGE MODEL (LM)

- Language models are algorithms that predict
  - ➢ probability of a sequence of words appearing in a sentence.
  - ➢ or probability of an upcoming word.



- n-gram

- Recurrent Neural Networks (RNNs)

- GPT (Generative Pre-trained Transformer) Series

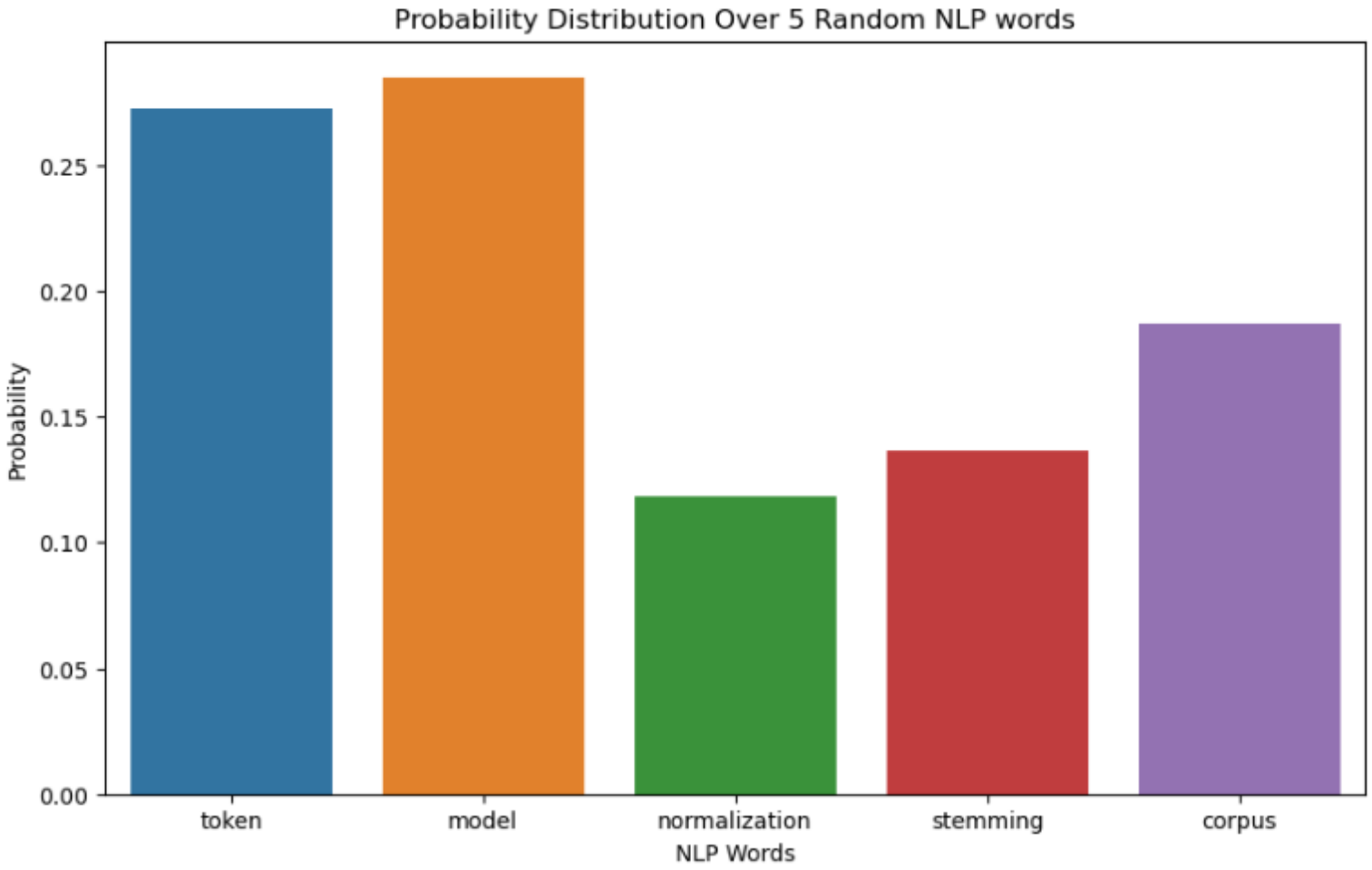
Do not edit
How to change the design

Predict the Next Word!

The Slido app must be installed on every computer you're presenting from

slido

A probability distribution over words



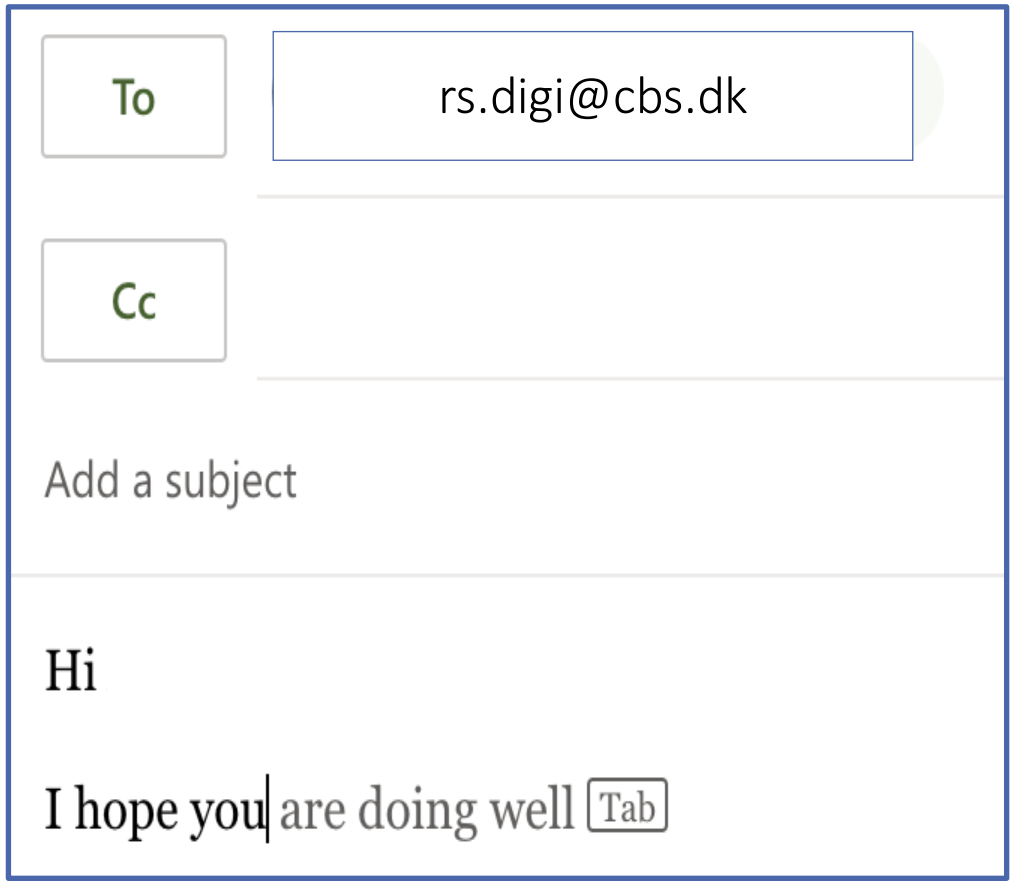
CBS

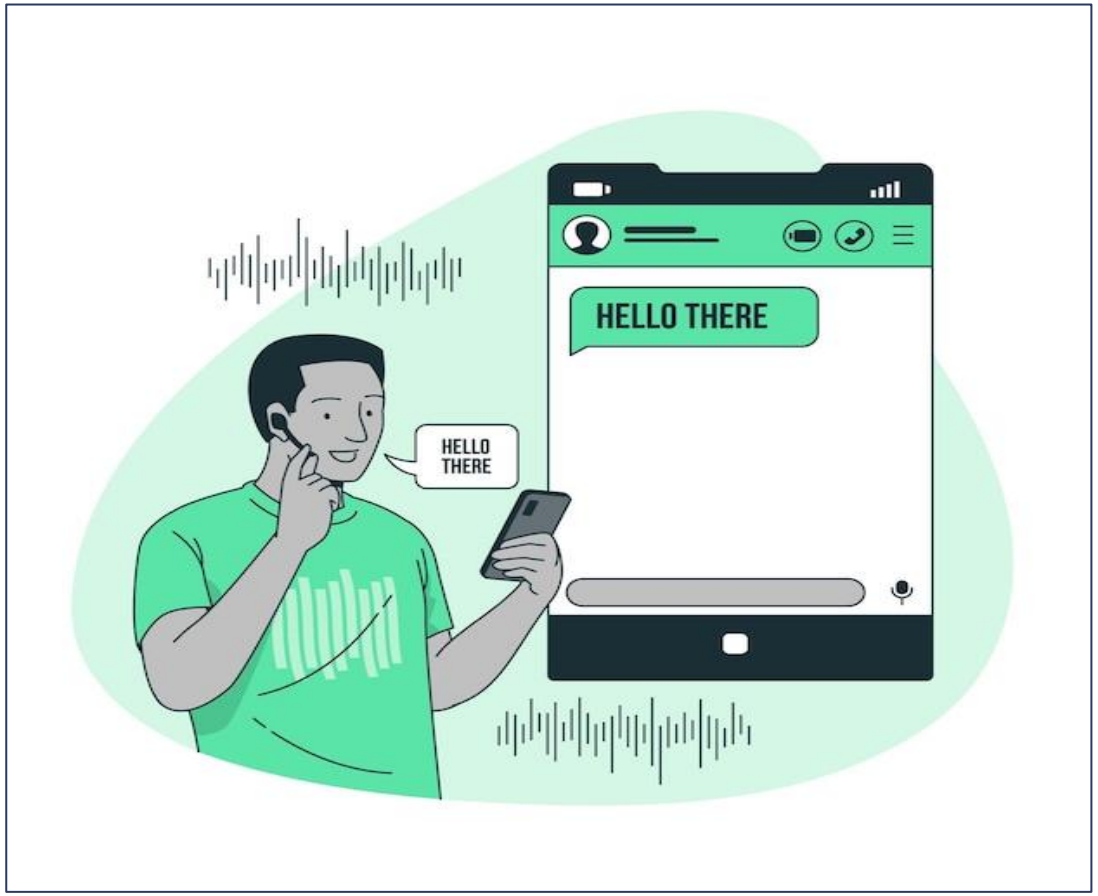
Application in NLP

Speech recognition	• *I ate a cherry* is a more likely than *Eye eightuh Jerry*
Machine translation	• Pr[*high winds tonight*] > Pr[*large winds tonight*]
Context sensitive spelling correctors	• *Their are problems wit this sentence.*
Sentence completion or word prediction	• *Please turn off your ….*

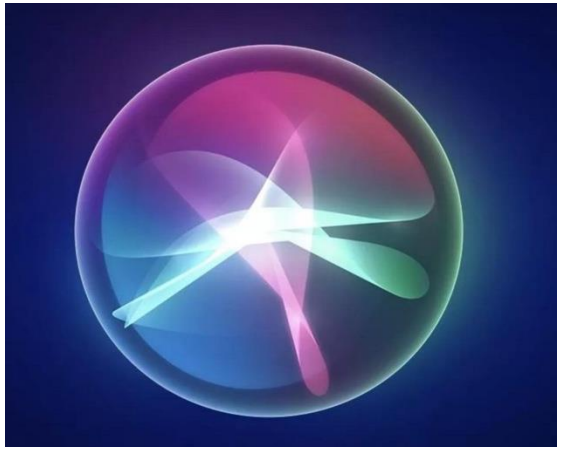
CBS

Practical Implementation of LM





Practical Implementation of LM



- Voice Recognition
- Task Automation
- Personalization
- Conversational Interaction
- Natural Language Understanding

Virtual assistants

Understanding N-grams

- An n-gram is a contiguous sequence of n items from a given sample of text or speech.
- Example: *Italy is famous for its cuisine.*

N-grams for sample text				
index	Unigram	Bigram	Trigram	Quadrigrams
0	('Italy',)	('Italy', 'is')	('Italy', 'is', 'famous')	('Italy', 'is', 'famous', 'for')
1	('is',)	('is', 'famous')	('is', 'famous', 'for')	('is', 'famous', 'for', 'its')
2	('famous',)	('famous', 'for')	('famous', 'for', 'its')	('famous', 'for', 'its', 'cuisine')
3	('for',)	('for', 'its')	('for', 'its', 'cuisine')	('for', 'its', 'cuisine', '.')
4	('its',)	('its', 'cuisine')	('its', 'cuisine', '.')	
5	('cuisine',)	('cuisine', '.')		
6	('.',)			

Your Turn

slido

List all the 5-grams in the sentence "The secret of business is to know something that nobody else knows."?

ⓘ Start presenting to display the poll results on this slide.

5-grams

index	Pentagrams
1	('The', 'secret', 'of', 'business', 'is')
2	('secret', 'of', 'business', 'is', 'to')
3	('of', 'business', 'is', 'to', 'know')
4	('business', 'is', 'to', 'know', 'something')
5	('is', 'to', 'know', 'something', 'that')
6	('to', 'know', 'something', 'that', 'nobody')
7	('know', 'something', 'that', 'nobody', 'else')
8	('something', 'that', 'nobody', 'else', 'knows')
9	('that', 'nobody', 'else', 'knows', '.')

N-gram Mechanism

Goal

- Compute the probability of a sentence
- $P(S) = P(w_1, w_2, w_3 \ldots w_k)$

Related task

- Compute the probability of an upcoming word
- $P(w_5 | w_1, w_2, w_3, w_4)$

Some basics on probability are needed!

Conditional Probability

How does the probability of an event change if we have extra information?

- Example: Toss a fair coin 3 times.

A. What is the probability of 3 heads?

Sample space Ω = {HHH, HHT, HTH, HTT, THH, THT, TTH, TTT }.

- All outcomes are equally likely, so P(HHH) = 1/8.

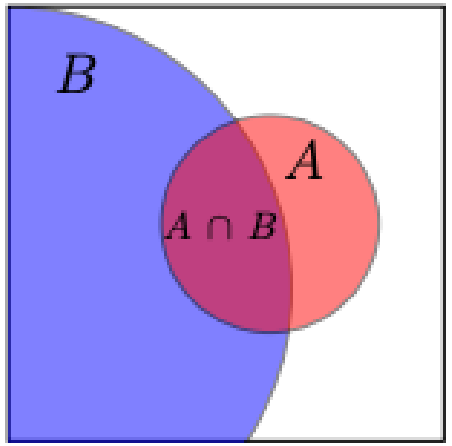
B. Suppose if we are told that the first toss was heads.

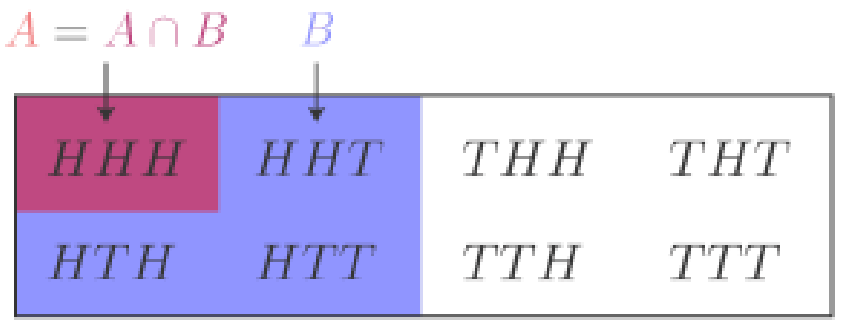
- Given this information how should we compute the probability of 3 heads?
  - New sample space = Ω' = {HHH, HHT, HTH, HT T}.
  - All outcomes are equally likely, so P(HHH|first toss is heads) = 1/4.
- This is called conditional probability, since it takes into account additional conditions.

CBS

Conditional Probability

The conditional probability of A knowing that B occurred is $P(A|B) = \dfrac{P(A \cap B)}{P(B)}$, $P(B) \neq 0$.





Event A = all three tosses are heads

Event B = the first toss was heads

Since $A \cap B = A$

$P(A) = 1/8$

$P(B) = 1/2$

$P(A \cap B) = 1/8$.

$$P(A|B) = \frac{1/8}{1/2} = \frac{1}{4}$$

Click here for more info

CBS

Chain Rule

Chain rule - computing the joint probability of multiple events by decomposing it into conditional probabilities

- Example: For two random events A and B:

$$P(A \cap B) \text{ or } P(A,B) = P(B|A) \,.\, P(A)$$

- The Chain Rule:

$$P(S) = P(w_1, w_2 \ldots w_k) = P(w_1) \,.\, P(w_2 | w_1) \,.\, P(w_3 | w_1, w_2) \ldots P(w_n | w_1 \ldots w_{k-1})$$

- Example: Sentence = I like NLP Course

$$P(\text{Sentence}) = P(\text{I}) * P(\text{like}|\text{I}) * P(\text{NLP}|\text{I like}) * P(\text{Course}|\text{I like NLP})$$

CBS

How to estimate probability?

Markov Assumption

Current state depends on only a finite fixed number of previous states.

- $P(w_k|w_1w_2...w_{k-1}) \approxeq P(w_k|w_{k-i},w_{k-(i-1)}...w_{k-1})$

  - Unigram:

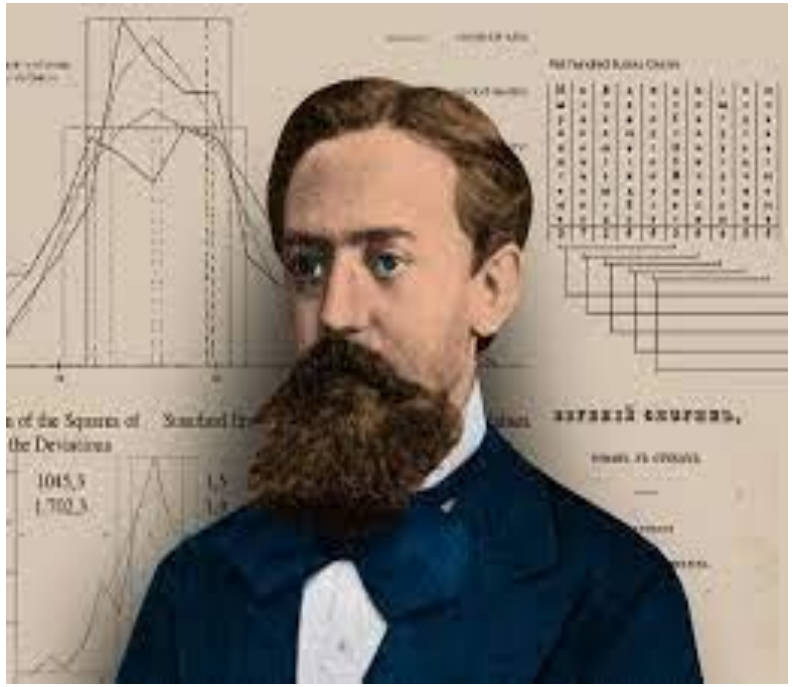
    $P(S) = P(I) \,.\, P(like) \,.\, P(NLP) \,.\, P(Course)$

  - Bigram:

    $P(S) = P(I) \,.\, P(like|I) \,.\, P(NLP|like) \,.\, P(Course|NLP)$

  - Trigram:

    $P(S) = P(I) \,.\, P(like|I) \,.\, P(NLP|I\ like) \,.\, P(Course|like\ NLP)$



Andrey Markov

Google N-gram = Click here

CBS

Probability of a word?

Unigram Model:	• $P(w_t) = \frac{\text{Count of word}}{\text{Total number of words}}$
Bigram Model:	• $P(w_t \mid w_{t-1}) = \frac{\text{Count of bigram}}{\text{Count of previous words}}$
Trigram Model:	• $P(w_t \mid w_{t-2}, w_{t-1}) = \frac{\text{Count of trigram}}{\text{Count of previous two words}}$

CBS

Estimating Probabilities in Bigram

What counts do I have to keep track of ?

- The Maximum Likelihood Estimate (MLE): $P(w_t | w_{t-1}) = \frac{\text{Count}(wt_{-1}, w_t)}{\text{Count}(wt_{-1})}$

Tokenization with Padding

- Add start and end tokens to each sentence.
- Split them into bigrams.

Counting

- Count the occurrences of each unique bigram.

Probability Calculation

- Calculate the probability of each bigram occurring.

CBS

Example: Estimating Bigram Probabilities

1. I like NLP.
2. NLP is fun.
3. NLP in python is fun.

1. <s> I like NLP. </s>
2. <s> NLP is fun. </s>
3. <s> NLP in python is fun. </s>

Step 1: Bigram tokenization with Padding

index	Bigrams
1	('<s>', 'I')
2	('I', 'like')
3	('like', 'NLP.')
4	('NLP.', '</s>')
5	('</s>', '<s>')
6	('<s>', 'NLP')
7	('NLP', 'is')
8	('is', 'fun.')
9	('fun.', '</s>')
10	('</s>', '<s>')
11	('<s>', 'NLP')
12	('NLP', 'in')
13	('in', 'python')
14	('python', 'is')
15	('is', 'fun.')
16	('fun.', '</s>')

CBS

Example: Estimating Bigram Probabilities

index	Bigrams
1	('<s>', 'I')
2	('I', 'like')
3	('like', 'NLP.')
4	('NLP.', '</s>')
5	('</s>', '<s>')
6	('<s>', 'NLP')
7	('NLP', 'is')
8	('is', 'fun.')
9	('fun.', '</s>')
10	('</s>', '<s>')
11	('<s>', 'NLP')
12	('NLP', 'in')
13	('in', 'python')
14	('python', 'is')
15	('is', 'fun.')
16	('fun.', '</s>')

Step 2: Counting

index	Unique Bigrams	Counts
1	('<s>', 'I')	1
2	('I', 'like')	1
3	('like', 'NLP.')	1
4	('NLP.', '</s>')	1
5	('</s>', '<s>')	2
6	('<s>', 'NLP')	2
7	('NLP', 'is')	1
8	('is', 'fun.')	2
9	('fun.', '</s>')	2
10	('NLP', 'in')	1
11	('in', 'python')	1
12	('python', 'is')	1

Example: Estimating Bigram Probabilities

- $P(w_t | w_{t-1}) = \frac{\text{Count}(wt_{-1}, w_t)}{\text{Count}(wt_{-1})}$

index	Unique Bigrams	Counts
1	('<s>', 'I')	1
2	('I', 'like')	1
3	('like', 'NLP.')	1
4	('NLP.', '</s>')	1
5	('</s>', '<s>')	2
6	('<s>', 'NLP')	2
7	('NLP', 'is')	1
8	('is', 'fun.')	2
9	('fun.', '</s>')	2
10	('NLP', 'in')	1
11	('in', 'python')	1
12	('python', 'is')	1

index	Unique Unigrams	Counts
1	<s>	3
2	I	1
3	like	1
4	NLP.	1
5	</s>	3
6	NLP	2
7	is	2
8	fun.	2
9	in	1
10	python	1

Step 3: Probability Calculation

Word 1	Word 2	MLE of Bigram
<s>	['I', 'NLP']	{'I': 0.33, 'NLP': 0.67}
I	['like']	{'like': 1.0}
like	['NLP.']	{'NLP.': 1.0}
NLP.	['</s>']	{'</s>': 1.0}
</s>	['<s>']	{'<s>': 0.67}
NLP	['is', 'in']	{'is': 0.5, 'in': 0.5}
is	['fun.']	{'fun.': 1.0}
fun.	['</s>']	{'</s>': 1.0}
in	['python']	{'python': 1.0}
python	['is']	{'is': 1.0}

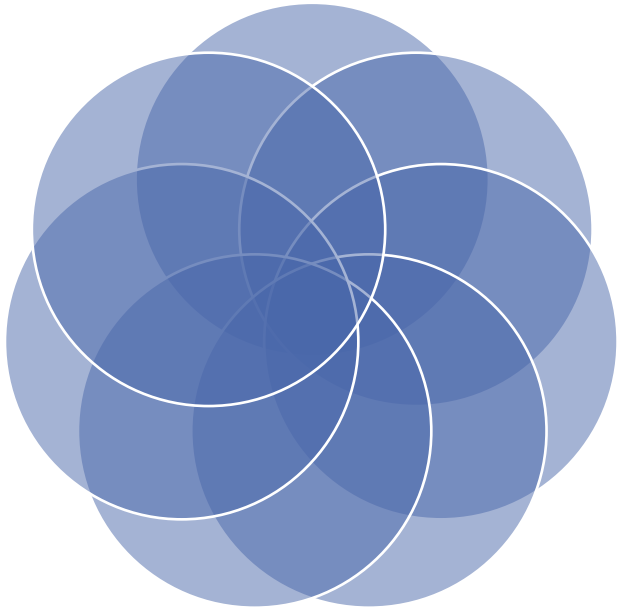
Total count of bigrams = 16

Application in NLP

Machine Translation

Question Answering

Text Summarization



Text Classification and Topic Modeling

Text Generation and Conversational AI

Sentiment Analysis and Opinion Mining

Named Entity Recognition (NER)

CBS

Why Padding?

- Important to pad each sentence with ”start” <s> and “end” </s> symbols
- Ex = I like NLP course --> <s> I like NLP course </s>
- But Why?
- Handling Variable-Length Sequences:
  - ensures that all sequences have the same length
  - crucial for tasks where sequences of different lengths need to be processed consistently
- Mitigating Boundary Effects :
  - tokens at the edges of sequences may lack context, leading to inaccurate predictions
  - adds context at the beginning and end of sequences

Issue of Underflow

P(i|<s>) = 0.25 (english|want) = 0.0011 P(food|english) = 0.5 P(</s>|food) = 0.68

P(<s> i want english food </s>) = P(i|<s>) × P(want|I) × P(english|want) × P(food|english) × P(</s>|food)
= 0.25 × 0.33 × 0.0011 × 0.5 × 0.68
= 0.000031

Underflow

How to avoid Underflow?

Solution: Do everything in log space

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Adding is faster than multiplying ☺

CBS

Issue of Zero Probability

- N-grams only work well for word prediction if the test corpus looks like the training corpus

Training set	Test set
3… denied the allegations 2… denied the reports	… denied the attack … denied the outcome P(attack| denied the) = 0

Wall Street Journal (WSJ) corpus from the Penn Treebank

How to solve zero probability problem?

Solution: Use smoothing techniques

Smoothing Techniques

1. Laplace Smoothing (Add-One Smoothing): Adds one to each count
2. Add-k Smoothing: A generalization of Laplace smoothing – add k>0 to each count.
3. Good-Turing Discounting:
   - Adjusts the counts of all n-grams, discounting the counts of observed n-grams and redistributing that probability mass to n-grams with zero counts.
4. Backoff and Interpolation:
   - These methods combine different n-gram models, using higher-order n-grams when the data is sufficient and backing off to lower-order n-grams when it is not.
5. Kneser-Ney Smoothing:

Smoothing Techniques

Smoothing Technique	Effect on Zero-Counts	Effect on Non-Zero Counts	Parameter Tuning Required	Typical Usage Scenario
No Smoothing	Probabilities remain zero	Unchanged	No	Baseline comparison
Add-One (Laplace)	Assigns non-zero probability	Decreases all non-zero counts	No	Small vocabulary tasks
Add-k	Assigns non-zero probability	Decreases all non-zero counts	Yes	Better for larger datasets
Good-Turing	Redistributes probabilities	Adjusts counts downward	Some	Small datasets with many rare events
Backoff	Uses lower-order n-grams	Adjusts based on context availability	Yes	Complex models with sufficient data
Interpolation	Combines estimates from different n-gram levels	Adjusts based on weighted combinations	Yes	Tasks requiring a balance between higher- and lower-order n-grams
Kneser-Ney	Assigns non-zero probability	Dynamic discounting based on context	Yes	Large datasets with sophisticated models

CBS

Laplace smoothing

- $P_{MLE}(w_t|w_{t-1}) = \frac{Count(wt_{-1}, w_t)}{Count(wt_{-1})}$ ⟹ $P_{LS}(w_t|w_{t-1}) = \frac{Count(wt_{-1}, w_t)+1}{Count(wt_{-1})+V}$

1. <s> I like NLP </s>
2. <s> NLP is fun </s>
3. <s> NLP in python is fun </s>

V= {<s>, I, like, NLP, </s>, is, fun, in, python}

Vocabulary Size (V) = 9

- For the bigram (<s> I):
  - Count(<s>, I) = 1
  - Count(<s>) = 3
- LS probability: P(I|<s>) = (1 + 1) / (3 + 9) = 2/12 = 1/6

- For the bigram (NLP </s>):
  - Count(NLP </s>) = 1
  - Count(NLP) = 3
- LS probability: P(</s>|NLP) = (1 + 1) / (3 + 9) = 2/12 = 1/6

CBS

Linear Interpolation

Combine different N-grams = unigram, bigram, trigram

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1 P(w_n|w_{n-2}w_{n-1}) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n)$$

$$\sum_i \lambda_i = 1$$

Training Data	Held-Out Data	Test Data

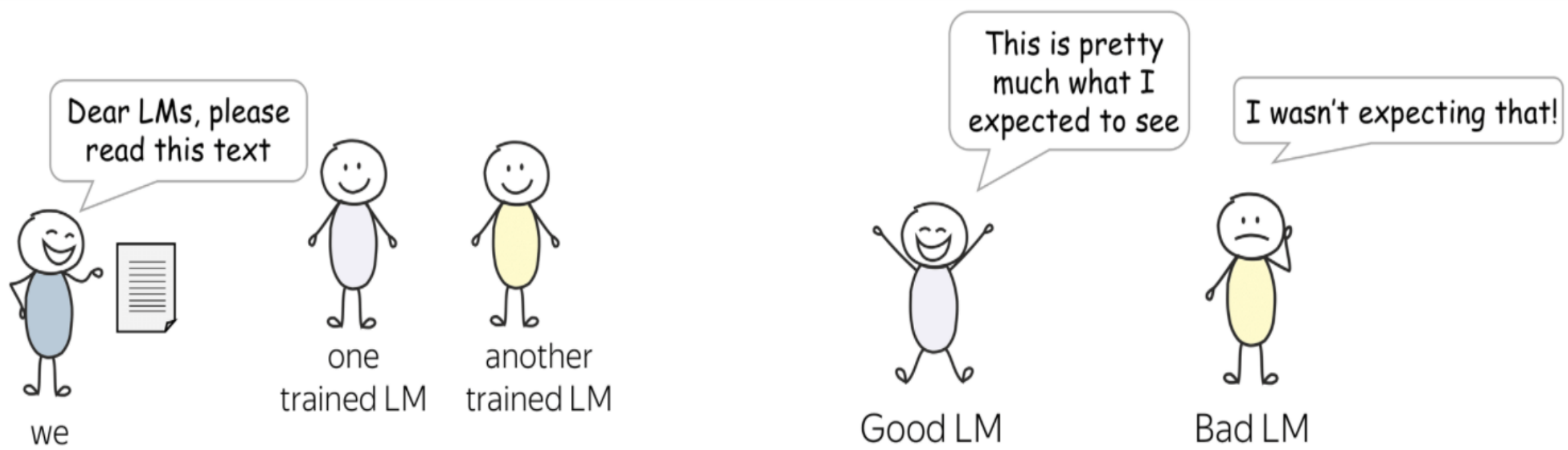
Choose λ's to maximize the probability of held-out data

- Fix the N-gram probabilities (on the training data)
- Search for λ's that give highest probability to held-out set

Evaluation = How good is our model?

Does our language model prefer good sentences to bad ones?

Assign higher probability to real or frequently observed sentences than ungrammatical or rarely observed sentences?



Extrinsic evaluation of N-gram models

Put each model in a task:

- spelling corrector, speech recognizer, MT system

Run the task, get an accuracy for both models and compare them

- How many misspelled words corrected properly?
- How many words translated correctly?

Problem! Time-consuming ; can take days or weeks

Intrinsic evaluation

- Train parameters of language model on a training set.
- Test the LM performance on test set.
  - - unseen dataset, different from training set, unused.
- Perplexity: Evaluation metric - tells us how well LM does on the test set.
- Problem! Bad approximation unless the test data looks just like the training data.

Perplexity

Perplexity is the inverse probability of the test set, normalized by the number of words:

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

(NB: Minimize Perplexity → Maximize probability of the sentence.)

Chain rule:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

For bigrams:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

CBS

Perplexity in Bigram

Example: The cat sat on the mat.

Probabilities for each bigram:

P(The | <s>) = 0.5
P(cat | The) = 0.2
P(sat | cat) = 0.1
P(on | sat) = 0.4
P(the | on) = 0.3
P(mat | the) = 0.2
P(</s> | mat) = 1.0

$$PP(W) = P(w_1, w_2, ..., w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1,w_2)\cdot P(w_2,w_3)\cdot ... \cdot P(w_{N-1},w_N)}}$$

where N is the number of words in the sentence (including `<s>` and `</s>`), and $P(w_{i-1}, w_i)$ is the probability of word w_i given its previous word w_{i-1}.

N=8

Perplexity=3.29

Key Takeaways:

Probability Estimation:

- essential for language-related predictions.

Smoothing Techniques:

- to deal with the issue of unseen n-grams in the training data

Data Sparsity and Dimensionality:

- Higher-order n-grams face data sparsity issues
- many possible combinations are often not observed in training data
- leading to computational inefficiency.

Perplexity:

- lower perplexity means better predictive power of the language model.

CBS

How Can I further help you?

CBS

Do not edit
How to change the design

Please fill in the survey

The Slido app must be installed on every computer you're presenting from

slido